Assignment Number:13.3
2303A51113 A.Harika B-26

Q.No.	Question	*Expected Time to complete*

1	**Lab 13: Code Refactoring Using AI Assistance Improving Legacy Code for Readability, Maintainability, and Performance**
	Lab Objectives
	The objectives of this laboratory exercise are to:
	• Identify common code smells and inefficiencies in legacy Python programs.
	• Use AI-assisted coding tools to refactor code for better readability and maintainability.
	• Apply modern Python best practices while preserving original program behavior.
	• Analyze performance improvements achieved through refactoring.
	Learning Outcomes
	After completing this lab, students will be able to:
	• Detect and eliminate code duplication in Python programs.
	• Refactor inefficient logic using optimized data structures and constructs.
	• Improve code quality through meaningful naming conventions and documentation.
	• Validate refactored code using test cases and performance comparison.
	Task 1: Refactoring – Removing Code Duplication
	Objective
	To eliminate repeated logic by extracting reusable functions.
	Task Description
	Use AI assistance to refactor a legacy Python script that contains repeated blocks of code calculating the area and perimeter of rectangles.
	Starter (Legacy) Code
	# Legacy script with repeated logic
	print("Area of Rectangle:", 5 * 10)
	print("Perimeter of Rectangle:", 2 * (5 + 10))
	print("Area of Rectangle:", 7 * 12)
	print("Perimeter of Rectangle:", 2 * (7 + 12))
	print("Area of Rectangle:", 10 * 15)
	print("Perimeter of Rectangle:", 2 * (10 + 15))
	Expected Outcome
	• A reusable function to calculate area and perimeter
	• No duplicated code blocks
	• Proper docstrings for all functions
	Prompt :
		Week7 - Wednesday

S Refactor the following legacy Python script to remove duplicated code. Create a reusable function that calculates both the area and perimeter of a rectangle, and call it for each set of dimensions. Add proper docstrings to the function explaining its purpose, parameters, and return values. Ensure the final script is concise, clear, and free of repeated logic. # Legacy script with repeated logic print("Area of Rectangle:", 5 * 10) print("Perimeter of Rectangle:", 2 * (5 + 10)) print("Area of Rectangle:", 7 * 12) print("Perimeter of Rectangle:", 2 * (7 + 12)) print("Area of Rectangle:", 10 * 15) print("Perimeter of Rectangle:", 2 * (10 + 15))

Code : Witgout refactoring

```
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))
print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))
print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

```
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
```

Refactored Code :

```python
def calculate_rectangle_properties(length, width):
    """
    Calculates the area and perimeter of a rectangle.

    Args:
        length (float or int): The length of the rectangle.
        width (float or int): The width of the rectangle.

    Returns:
        tuple: A tuple containing:
            - area (float or int): The area of the rectangle.
            - perimeter (float or int): The perimeter of the rectangle.
    """
    area = length * width
    perimeter = 2 * (length + width)
    return area, perimeter

# Dimensions for the rectangles
dimensions = [
    (5, 10),
    (7, 12),
    (10, 15)
]

# Calculate and print properties for each rectangle
for length, width in dimensions:
    area, perimeter = calculate_rectangle_properties(length, width)
    print(f"For a rectangle with length {length} and width {width}:")
    print(f"  Area: {area}")
    print(f"  Perimeter: {perimeter}")
    print("-" * 30)
```

```
For a rectangle with length 5 and width 10:
  Area: 50
  Perimeter: 30
------------------------------
For a rectangle with length 7 and width 12:
  Area: 84
  Perimeter: 38
------------------------------
For a rectangle with length 10 and width 15:
  Area: 150
  Perimeter: 50
------------------------------
```

Explanantion ;

1. **Duplication vs. Reusability (DRY Principle):**

Original: The formulas for calculating area (length * width) and perimeter (2 * (length + width)) are repeated three times. If you wanted to calculate properties for a fourth rectangle, you'd copy and paste the same logic again.The code is functional but can become cluttered and harder to follow as the number of rectangles (or operations) grows.

Refactored: The formulas are encapsulated within a single function calculate_rectangle_properties. This function is then called multiple times with different inputs. This adheres to the **D**on't **R**epeat **Y**ourself (DRY) principle. The function name calculate_rectangle_properties clearly describes its purpose. The main loop is also clear: it iterates through dimensions and calls the descriptive function

Task 2: Refactoring – Optimizing Loops and Conditionals

Objective

To improve performance by replacing inefficient nested loops with optimized structures.

Task Description

Use AI to analyze and refactor a script that checks the presence of elements using nested loops.

Starter (Legacy) Code

```
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]
for s in search_names:
found = False
for n in names:
if s == n:
found = True
if found:
print(f"{s} is in the list")
else:
print(f"{s} is not in the list")
```

Expected Outcome

- Optimized solution using set lookups or comprehensions
- Performance comparison before and after refactoring

Prompt :

Refactor this Python script to replace inefficient nested loops with optimized structures. Use set lookups or comprehensions to check membership instead of looping through lists. Add comments or docstrings explaining the improvement, and compare performance before and after refactoring." names = ["Alice", "Bob", "Charlie", "David"] search_names = ["Charlie", "Eve", "Bob"] for s in search_names: found = False for n in names: if s == n: found = True if found: print(f"{s} is in the list") else: print(f"{s} is not in the list")

Code : Without refactoring

```
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]

for s in search_names:
    found = False
    for n in names:
        if s == n:
            found = True
    if found:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")
```

```
Charlie is in the list
Eve is not in the list
Bob is in the list
```

Refactored Code :

```
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]

# Refactored: Convert 'names' to a set for O(1) average time complexity lookups.
# This avoids the inefficient O(N) lookup for each item in 'search_names' within the inner lo
names_set = set(names)

for s in search_names:
    if s in names_set: # Efficient O(1) average time complexity lookup
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")
```

```
Charlie is in the list
Eve is not in the list
Bob is in the list
```

Explanation : The core difference lies in how membership is checked. The first code brute-forces a search through a list for each item, while the second leverages the optimized data structure of a set for near-instant lookups, leading to a much more scalable and performant solution for larger datasets.

Task 3: Refactoring – Extracting Reusable Functions
Objective
To modularize code by extracting calculations into reusable functions.
Task Description
Refactor a legacy script where price and tax calculations are written inline.
Starter (Legacy) Code

```
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)

price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
```

Expected Outcome

- A function calculate_total(price)
- Cleaner and reusable code structure
- Proper documentation

Prompt :

```
Refactor this Python script to extract the repeated price and tax calculations into a reusable function calculate_total(price). Add proper docstrings explaining the function's purpose, parameters, and return value. Ensure the final code is cleaner, modular, and free of duplicated logic. price = 250 tax = price * 0.18 total = price + tax print("Total Price:", total) price = 500 ta
= price * 0.18 total = price + tax print("Total Price:", total)
```

Code : Without refactoring

```
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)

price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
```

```
Total Price: 295.0
Total Price: 590.0
```

With Refactoring :

```
def calculate_total(price):
    """
    Calculates the total price after adding a fixed tax rate of 18%.

    Args:
        price (float or int): The base price of an item.

    Returns:
        float: The total price including tax.
    """
    tax_rate = 0.18
    tax = price * tax_rate
    total = price + tax
    return total

# Calculate and print total prices for different base prices
price1 = 250
total1 = calculate_total(price1)
print(f"Original Price: {price1}, Total Price with Tax: {total1}")

price2 = 500
total2 = calculate_total(price2)
print(f"Original Price: {price2}, Total Price with Tax: {total2}")
```

```
Original Price: 250, Total Price with Tax: 295.0
Original Price: 500, Total Price with Tax: 590.0
```

Explanantion : **Original:** The set of calculations (tax = price * 0.18, total = price + tax) is repeated verbatim for each price. If you had 10 different prices, you'd repeat this block 10 times.
Refactored: The calculation logic is placed inside a function calculate_total. This function is then *called* for each price, reusing the same code block. This adheres to the **D**on't **R**epeat **Y**ourself (DRY) principle.

Task 4: Refactoring – Replacing Hardcoded Values with Constants

Objective

To improve maintainability by replacing magic numbers with named constants.

Task Description

Use AI to identify hardcoded values and replace them with constants.

Starter (Legacy) Code

```
print("Area of Circle:", 3.14159 * (7 ** 2))
print("Circumference of Circle:", 2 * 3.14159 * 7)
```

Expected Outcome

- Constants such as PI and RADIUS
- Cleaner, maintainable code

Prompt :

```
print("Area of Circle:", 3.14159 * (7 ** 2)) print("Circumference of Circle:", 2 * 3.14159 * 7) Refactor this Python script to replace hardcoded values with named constants. Define constants such as PI and RADIUS, and use them in calculations for area and circumference. Add proper documentation to make the code cleaner and more maintainable."
```

Code :

```
print("Area of Circle:", 3.14159 * (7 ** 2))
print("Circumference of Circle:", 2 * 3.14159 * 7)
```

```
Area of Circle: 153.93791
Circumference of Circle: 43.98226
```

+ Code + Text

Refactored :

```
# Define constants for circle calculations
PI = 3.14159
RADIUS = 7

# Calculate and print the Area of the Circle
# Formula: Area = PI * radius^2
print("Area of Circle:", PI * (RADIUS ** 2))

# Calculate and print the Circumference of the Circle
# Formula: Circumference = 2 * PI * radius
print("Circumference of Circle:", 2 * PI * RADIUS)
```

```
Area of Circle: 153.93791
Circumference of Circle: 43.98226
```

Explanation : The script in cell 1Dx2R4bfHfcA to use named constants for PI and the radius. This makes the code more readable, maintainable, and easier to modify if these values ever need to change. I'll also add comments to explain the constants and formulas. The refactored script in cell 1Dx2R4bfHfcA executed successfully!

Task 5: Refactoring – Improving Variable Naming and Readability
Objective

To enhance readability using descriptive variable names and comments.

Task Description

Refactor a script with unclear variable names.

Starter (Legacy) Code

```
a = 10
b = 20
c = a * b / 2
print(c)
```

Expected Outcome

- Descriptive variable names
- Inline comments explaining logic
- Identical output

Prompt :

```
a = 10 b = 20 c = a * b / 2 print(c) Refactor this Python script to
replace unclear variable names with descriptive ones. Add
inline comments explaining the logic, while keeping the output
identical.
```

Code :

```
a = 10
b = 20
c = a * b / 2
print(c)
```

```
100.0
```

Refactored Code :

```
base = 10  # Define the base of the triangle
height = 20 # Define the height of the triangle

triangle_area = base * height / 2  # Calculate the area of the triangle (base * height / 2)
print(triangle_area)
```

```
100.0
```

Explanation : Clearer variable names and comments reduce the cognitive load when revisiting the code later, making it easier to maintain, debug, or extend. If you need to change the base or height, it's immediately obvious which variables to adjust. This refactoring improves the overall quality of the code by making it more human-readable and understandable, without changing its functionality.